Eclipse Attacks on Ethereum's Peer-to-Peer Network

Ruisheng Shi*
Beijing University of Posts and Telecommunications
Beijing, China
shiruisheng@bupt.edu.cn

Yuxuan Liang
Beijing University of Posts and Telecommunications
Beijing, China
dzqlyx@bupt.edu.cn

Zijun Guo
Beijing University of Posts and Telecommunications
Beijing, China
guozijunn@foxmail.com

Qin Wang
UNSW Sydney
Sydney, Australia
qinwangtech@gmail.com

Lina Lan*
Beijing University of Posts and Telecommunications
Beijing, China
lanlina@bupt.edu.cn

Chenfeng Wang
Beijing University of Posts and Telecommunications
Beijing, China
klaypexwcf@bupt.edu.cn

Zhuoyi Zheng
Beijing University of Posts and Telecommunications
Beijing, China
2025111069@bupt.cn

Abstract

Eclipse attacks isolate blockchain nodes by monopolizing their peer-to-peer connections. The attacks were extensively studied in Bitcoin (SP'15, SP'20, CCS'21, SP'23) and Monero (NDSS'25), but their practicality against Ethereum nodes remains underexplored, particularly in the post-Merge settings.

We present the first end-to-end implementation of an eclipse attack targeting Ethereum (2.0 version) execution-layer nodes. Our attack exploits the bootstrapping and peer management logic of Ethereum to fully isolate a node upon restart. We introduce a multi-stage strategy that majorly includes (i) poisoning the node's discovery table via unsolicited messages, (ii) infiltrating Ethereum's DNS-based peerlist by identifying and manipulating the official DNS crawler, and (iii) hijacking idle incoming connection slots across the network to block benign connections. Our DNS list poisoning is the first in the cryptocurrency context and requires only 28 IP addresses over 100 days. Slots hijacking raises outgoing redirection success from 45% to 95%. We validate our approach through controlled experiments on Ethereum's Sepolia testnet and broad measurements on the mainnet. Our findings demonstrate that over 80% of public nodes do not leave sufficient idle capacity for effective slots occupation, highlighting the feasibility and severity of the threat. We further propose concrete countermeasures and responsibly disclosed all findings to Ethereum's security team.

*Corresponding authors



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792231

CCS Concepts

• **Security and privacy** → **Network security**; **Pseudonymity, anonymity and untraceability**.

Keywords

Eclipse Attacks, Ethereum, Web3, Peer-to-Peer Network

ACM Reference Format:
Ruisheng Shi, Yuxuan Liang, Zijun Guo, Qin Wang, Lina Lan, Chenfeng Wang, and Zhuoyi Zheng. 2026. Eclipse Attacks on Ethereum's Peer-to-Peer Network. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3774904.3792231

1 Introduction

Ethereum has become one of the most successful cryptocurrency platforms. On Sep. 15, 2022, Ethereum completed its long-anticipated merge [1], transitioning from Proof-of-Work (PoW) to Proof-of-Stake (PoS). This merge unified the original Ethereum mainnet (execution layer) and the Beacon Chain (consensus layer), reducing energy consumption by 99.95% by eliminating the need for energy-intensive mining. This transition introduced new assumptions and behavioural changes in the peer-to-peer (P2P) network layer [2]:

- **Long-lived validator nodes.** PoS validators maintain persistent online presence, resulting in more stable peer sets.
- **Static bootstrapping and peer discovery.** Validators often reuse static peer configurations or depend heavily on DNS-based discovery mechanisms.
- **Client usage patterns.** A small number of client pairings (e.g., Prysm+Geth) dominate validator deployments.
- **Exposure of execution clients.** Although block finality is determined by the consensus layer, the execution client handles transaction gossip and mempool operations.

These shifts call for a renewed assessment of classical network-layer threats within Ethereum's PoS architecture. For instance, the persistent online presence of validators results in stable peer sets,

making them ideal targets for long-term network manipulation. The reliance on static bootstrapping and DNS-based discovery significantly enhances the success rate of peer poisoning strategies.

In this paper, we revisit and refine the classic eclipse attack under these updated assumptions. We propose a practical, multi-stage attack that leverages previously discussed vectors (e.g., discovery table poisoning, DNS list infiltration) to strategically occupy both incoming and outgoing connections.

We present our progress in a step-by-step manner.

▹ **We systematically analyze the P2P networking behavior of post-Merge Ethereum execution-layer nodes.**

We deeply analyze how post-Merge Ethereum nodes discover peers, manage connections, and maintain the network. Our study covers both client architecture and address formats, particularly the transition from legacy enode to the more extensible ENR format. We delve into the structure and management of both short-term (discovery table) and long-term (DB) peer records, highlighting mechanisms such as Discv4 message handling, dynamic K-bucket updates, IP-based admission limits, and initialization routines that protect against adversarial influence.

In addition, we explore the full connection management lifecycle, including the establishment and limitations of incoming and outgoing connections, rate-limiting policies, and the fallback to DNS-based discovery. We further examine the role of the `devp2p` crawler in maintaining Ethereum's DNS peerlist, detailing its scoring system, update frequency, and pruning behavior. These insights reveal points of vulnerability within Ethereum's peer discovery and connection protocol stack, which inform our attack design and defense proposals.

▹ **We propose the first practical eclipse attack against Ethereum's post-Merge P2P network.**

Eclipse attacks isolate a target node by controlling all its P2P connections [3–6]. Once isolated, an attacker can feed the victim false information, undermining blockchain consistency and enabling various attacks—such as deanonymization [7–9], selfish mining [10–13], double spending, or smart contract manipulation [14]. These attacks reduce network resilience and can support larger threats like 51% attacks.

Research on Ethereum's network layer has explored its security [15], topology [16, 17], and P2P behavior [18], and attacks such as partitioning and eclipsing [6, 19–23], topology mapping [16, 17, 24–33], and network-layer deanonymization [18, 34–40]. However, most eclipse attack studies focus on Bitcoin [41–43] or Monero [44], and existing Ethereum work largely predates the Merge [19–21].

▹ **We have new designs for implementing the attack.**

Ethereum's current network stack introduces new challenges for eclipse attacks. Recent Geth versions preload benign peers into the discovery table, blocking early attacker entries. Since v1.9.11, Ethereum also uses DNS-based peer discovery, which can bypass poisoned tables. Finally, each node only accepts 34 incoming connections, making slot preemption essential.

We design a multi-stage attack. We first fill the target's peerlist (discovery table and DNS), then hijack incoming slots network-wide. When the target restarts, it is forced to connect only to attacker-controlled nodes so as to complete the eclipse.

Here, we highlight our new designs.

▹ **We evaluate attacks on both Sepolia testnet and mainnet.**

We implement and evaluate our attack pipeline on Sepolia (testnet), validating the feasibility and success rate of each attack stage under realistic network conditions. To assess applicability, we also analyze Ethereum mainnet nodes and measure network-wide slot availability, connection patterns, and peer behavior.

We conduct a step-by-step experimental analysis of each attack component. For instance, in evaluating the incoming slot hijacking strategy, we scan over 2000 public Ethereum nodes and observe that more than **80%** of them have at most **10** available incoming slots, confirming the feasibility of large-scale slots occupation. Our DNS poisoning strategy is tested over a 3-month period, during which we successfully infiltrate the DNS list with attacker-controlled IPs. Experiments are carefully documented, and key metrics (e.g., filling rates, IP resource usage, timing thresholds) are quantified in §4.

▹ **We take further responsible actions.**

To promote the security of the Ethereum ecosystem, we have taken responsible steps throughout the course of this work.

> All our identified vulnerabilities and attack workflow were responsibly disclosed to Ethereum official team.

Alongside these disclosures, we proposed concrete mitigation strategies, including immediate implementation-level patches as well as longer-term protocol design improvements.

Last, we summarized our contributions in short:

- full exploration on Ethereum's P2P network (§2);
- a new practical, multi-stage eclipse attack (§3);
- evaluation on testnet and mainnet (§4);
- countermeasure suggestions (§6).

2 Dissecting Ethereum P2P Network

We analyze the network-layer behavior of Ethereum nodes based on `Geth v1.14.3` [45] (released May 9, 2024), focusing on: (i) node roles/types; (ii) peerlist management; (iii) connection handling and (iv) DNS peerlist management.

2.1 Ethereum Nodes

An Ethereum node consists of two coordinated clients: the *execution*[1] client and the *consensus* client [46]. The execution client handles transaction propagation, while the consensus client is responsible for block propagation and participation in the PoS consensus protocol. Each client connects to their P2P network.

2.2 Peerlist Management

To support efficient node discovery and connection establishment, Ethereum employs a distributed and dynamically updated peerlist management mechanism.

Short-&long-term databases. The local node maintains a long-term node storage database (referred to as *DB*) with no size limitations. This database persists even after the local node is restarted, retaining all stored nodes. Concurrently, each time the local node starts up, it creates a short-term node database in the memory called the *discovery table*, used for node discovery and connection

[1]This paper focuses exclusively on the execution client. Unless otherwise stated, references to an "Ethereum node" refer to the execution client.

management within the Ethereum network. Unlike the long-term database, this short-term one is cleared entirely upon each restart of the local node.

Discv4 messages. Ethereum's node discovery is implemented via the Discv4 protocol [47], which operates over UDP. The protocol defines six types of message packets, including `Ping`/`Pong` for liveness detection and node table updates, `FindNode`/`Neighbors` for locating the closest nodes in the ID space, and `ENRRequest`/`ENRResponse` for exchanging node records. Together, these messages enable nodes to test connectivity, discover new peers, and maintain an up-to-date view of the network topology.

Management mechanism of *DB*. The long-term database (i.e., *DB*) maintains metadata for each discovered node. Two main operations are performed:

- *Node addition:* A node is added to the database if it has remained in the discovery table for over 5 minutes and has passed at least one liveness check. This check is performed in a recurring cycle every 30 seconds.
- *Node deletion:* Nodes that have not responded to `Ping` messages within the last 24 hours (as indicated by the `lastpong` timestamp) are removed from the database. This cleanup process is triggered every hour.

Data structure of discovery table. The discovery table comprises 17 K-buckets, each structured according to the Kademlia protocol to manage and organize peer information in a P2P network. Nodes are grouped based on their XOR distance from the local node and are ranked by activity. Since every bucket stores up to 16 active nodes, the table can accommodate up to 272 ($= 17 \times 16$) nodes.

To prevent overrepresentation of nodes from the same subnet, Ethereum enforces two IP-level restrictions:

- `bucketIPLimit:` At most 2 nodes from the same /24 subnet may reside in any single bucket.
- `tableIPLimit:` Across the entire discovery table, a maximum of 10 nodes from the same /24 subnet is allowed.

Initialization of discovery table. After the local node starts, the table is initialized through the following steps:

- The local node first loads 30 random nodes from the database and adds 4 bootnodes to form 34 seed nodes (via `loadSeedNodes`). "Random" means generating an ID and selecting the first node in the database with a greater or equal lexicographic ID.
- It then performs a self-lookup and a random lookup to populate the table with benign nodes (via `doRefresh`, §2.2), reducing the risk of attacker-controlled entries.

Once initialization is complete, the discovery table enters a dynamic update phase, during which it continuously refreshes its entries based on network activity and liveness checks.

Update of discovery table. The discovery table is updated via node insertion, deletion, and replacement.

- *Node filling.* The discovery table is populated through both active and passive mechanisms. In the active case, the node periodically runs the `doRefresh` function (every 15–30 minutes) to insert seed nodes (`loadSeedNodes`, see §2.2) and perform three random lookups (`lookupRandom`) to refresh bucket entries. Each lookup initiates a recursive `FindNode` query based on a randomly generated target ID, collecting `Neighbors` responses to progressively discover closer nodes, which are then stored in the table. In the passive case, the node adds peers that send it `Ping` messages during normal operation (lookup recursive search refers to Appendix B).
- *Node deletion and replacement.* The `doRevalidate` function runs every 0–10 seconds, randomly selecting a bucket to verify liveness. It pings the latest seen node; if responsive, the node is promoted within the bucket and its ENR is updated if needed. If unresponsive, it is replaced by a node from the replacement list.

2.3 Connections Management

By default, the upper bound of connections for an Ethereum node is 50, with a maximum of 16 outgoing and 34 incoming connections.

Incoming connections establishment. An Ethereum node allows up to 34 incoming connections. Since incoming connections are passively established, we discuss the reasons why such connection attempts may fail in Appendix C.

Outgoing connections establishment. A node maintains up to 16 outgoing connections. After startup, once the discovery table is initialized, the node begins establishing outgoing connections via a dial management loop, which continuously attempts to connect to peers if slots are available. Static nodes (manually configured peers) are prioritized but are not considered here as they need to be added manually by users. Peers are selected with equal probability from two sources: lookup discovery and DNS discovery. See more details of establishing outgoing connections in Appendix D.

Connections dropping. Dropped connections in the network are primarily caused by network-layer issues, such as TCP connection timeouts or peer nodes going offline.

2.4 Devp2p Tool Management

Ethereum uses the `devp2p` command-line tool, embedded in the Geth client, to maintain the official DNS peerlist. Nodes listed here only support the UDP-based discovery protocol and are crawled periodically by the `devp2p` tool.

Crawler. Each `devp2p` node concurrently performs: (i) updates to nodes listed in the input JSON file, and (ii) iterative peer discovery from seed nodes by issuing `FindNode` queries to known peers.

Scoring policy. When a `devp2p` crawler updates a node's ENR via a successful `ENRResponse`, the node gains +1 point. Crawlers throttle ENR updates to one every 10 minutes. If a node fails to respond, its score is halved. Once the score drops below 0, the node is evicted from DNS list. Specifically, the devp2p tool awards points to crawled nodes in two scenarios:

- *Startup update bonus.* When a crawl begins, the devp2p node updates all nodes listed in the DNS peerlist. Any node that is found to be online receives +1 point.
- *Node discovery bonus.* During the crawl, the devp2p node selects peers from its discovery table and sends them FindNode requests. Any node returned in the Neighbors response is also awarded +1 point.

Crawling schedule. Ethereum launches a `devp2p` crawler every 5 hours and 40 minutes. Each crawl lasts 30 minutes, repeating 4–5 times daily. Nodes from all crawls are merged into an aggregate

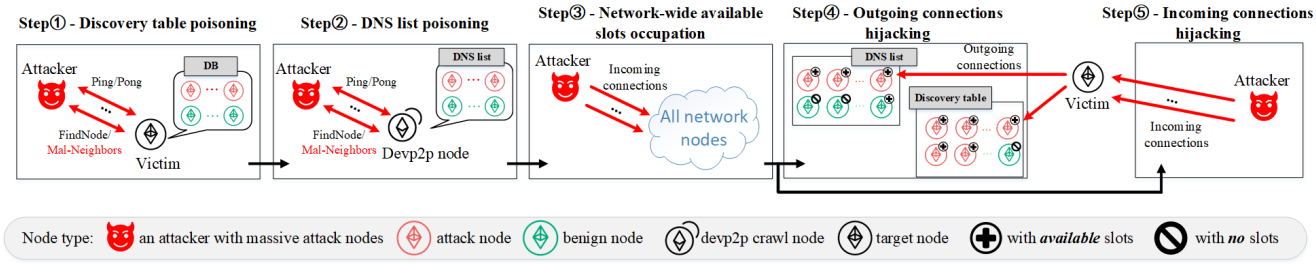


Figure 1: Our Eclipse Attack Workflow (Steps ①–③ operate in parallel in practice, we illustrate them sequentially for clarity.)

list (`all.json`), from which the top-N nodes (based on score) are selected per network – 3000 for mainnet, 250 for testnet.

3 Our Eclipse Attack

We target Ethereum execution layer nodes with public IP addresses, occupying incoming and outgoing connections of the target node.

3.1 Overview of The Attack

Our attack proceeds in five steps (illustrated in Fig. 1). We present details for each step in the following sections.

① **Discovery table poisoning by *DB* pre-filling.** Before the target node launches, an attacker pre-fills its local database (*DB*) to increase the probability of malicious entries being loaded into the table upon startup.

② **DNS list poisoning.** An attacker continuously injects malicious ENRs into Ethereum's DNS-based discovery list prior to target node's restart, polluting a secondary peer source.

③ **Network-wide available slots occupation.** The attacker preemptively fills available incoming connection slots across the network, reducing the number of reachable honest peers when the target node attempts to connect after reboot.

④ **Outgoing connections hijacking.** Upon the target node's restart, the attacker activates numerous malicious nodes to send `Ping` messages, filling the discovery table. With most honest nodes' incoming slots already occupied, the target node is likely to establish outgoing connections with attacker-controlled peers.

⑤ **Incoming connections hijacking.** Simultaneously, the attacker floods the target node with incoming connection requests, exhausting incoming connection limits and completing the eclipse.

3.2 (Step-①) Discovery Table Poisoning

We begin our attack by pre-filling the target node's persistent database (i.e., *DB*) to poison its discovery table upon restart. Normally, when a node boots up, it initializes its discovery table with a small set of seed nodes—some randomly selected from *DB*, and others hardcoded as bootnodes. As a result, even before the node starts processing unsolicited `Ping` messages, part of its discovery table is already occupied by these entries. This limits the attacker's ability to fully control the table post-restart.

To overcome this limitation, we launch a two-stage attack. First, we inject attacker-controlled nodes into the target's *DB* before it restarts. Then, upon the next boot, these nodes are likely selected as seed nodes, effectively giving the attacker a foothold in the discovery table from the outset.

To achieve DB poisoning, we send `Ping` messages from attacker nodes to the target node, leveraging the passive discovery behavior (§2.2), which causes the target to insert those nodes into its discovery table. Then, we rely on the node addition logic of the persistent database (§2.2), which periodically stores discovery table entries that have passed liveness checks for more than 5 minutes into *DB*.

To maximize poisoning coverage, we perform the attack in multiple rounds, described below:

- We detected the restart of the target node.
- We immediately send `Ping` messages from the first round of attacker nodes, which gradually populate both the discovery table and *DB*.
- Once the first-round nodes are stored in *DB*, we terminate them and launch a second round of attacker nodes. As the first-round nodes go offline, the second-round nodes replace them in the discovery table and eventually in *DB*.
- We repeat this replacement process across multiple rounds until *DB* is sufficiently filled with attacker-controlled ENRs.
- Finally, we detect the restart of the target node for a second time, triggering a discovery table initialization phase dominated by attacker entries drawn from the poisoned database *DB*.

This secondary restart is essential. It ensures that the attack nodes from the poisoned *DB* are loaded early into the discovery table, improving the attacker's success rate. Although the first restart is needed to initiate the initial discovery table state, subsequent rounds do not require restarts, as discovery table entries can still be updated efficiently while the node is online.

During the target node's second restart, we launch a discovery table poisoning attack concurrently. Specifically, 272 attacking nodes (matching the 272 slots in the discovery table) send `Ping` messages to the target node to maximize attacker's occupancy of the discovery table. To further improve the occupancy, we modify the behavior of the attacking nodes: upon receiving `FindNode` messages from the target node, they respond with `Neighbors` messages that include attacking nodes rather than benign nodes.

3.3 (Step-②) DNS List Poisoning

Next, we pre-fill the Ethereum DNS discovery list with attacker-controlled nodes, such that the target node is likely to select our malicious nodes when establishing outgoing connections. Our attack proceeds in two main phases.

We first inject malicious nodes into the master DNS list. We launch a large number of attacker nodes and keep them stably online. Empirically, a mainnet node is likely to be included in the DNS list within 1–2 days, while a Sepolia testnet node takes about 10 days. To improve visibility, we configure nodes with low ENR node IDs (e.g., prefixed with leading zeros) so that they are ranked at the top of crawler input list.

After our nodes are included, we repeatedly trigger score increments to improve their ranking:

- Identify the official `devp2p` crawler node;
- Pre-generate node IDs prefixed with zeros and start nodes to appear first in the crawler's input file;
- Monitor incoming `ENRRequest` messages to infer the crawler's launch time;
- Activate multiple malicious nodes to `Ping` the crawler, fill its discovery table, and respond to `FindNode` queries with neighbors lists containing our own malicious nodes;
- Respond to any `ENRRequest` from the crawler to gain score increments.

Once a sufficient number of our nodes reach high scores, they are selected into the top-N official DNS list used by Geth clients at startup, thereby increasing the probability that the target node connects to malicious peers.

3.4 (Step-③) Network-wide Slots Occupation

We preemptively occupy the available incoming connection slots of as many benign nodes as possible. This blocks the target node from establishing connections with honest peers; when the target attempts to connect, it is forced to rely on attacker-controlled nodes instead—enabling an eclipse attack.

3.5 (Step-④) Outgoing Connections Hijacking

After the target node restarts, all of its existing connections are dropped. We take advantage of this reset: since Ethereum lacks an active connection eviction policy, we activate a swarm of attacker nodes that rapidly send `Ping` messages to the target. This increases the chances that the target will select these attacker nodes from its discovery table or DNS list for outgoing connections.

3.6 (Step-⑤) Incoming Connections Hijacking

Upon the target node restarts, its incoming connection slots are unoccupied. We launch a large number of malicious nodes that aggressively initiate connection requests to the target, aiming to fill all of its incoming connection slots before benign peers connect.

4 Evaluation of Our Attack

We implemented our eclipse attack with details deferred to Appendix F. Then, we conducted experimental evaluation towards each attack strategy in §3.

4.1 Evaluating Step-① (Discovery Table)

We first investigated the attack targeting Ethereum's node discovery mechanism. In this strategy, an attacker preemptively sends a large number of crafted node records to potential victim nodes, aiming to insert and maintain attacker-controlled entries in their discovery tables. This manipulation increases the likelihood of the attacker being selected during peer connections, which is the foundation for our eclipse attack. All experiments in this part were conducted on Sepolia testnet.

We begin by measuring the growth scale and storage characteristics of node database to assess the resource cost and practicality of such an attack. Next, we design and validate a feasible filling strategy that complies with the protocol while continuously pushing malicious records to the target. Finally, we conduct controlled experiments to evaluate the effectiveness of the attack, focusing on the proportion of attacker nodes in the victim's discovery table.

Our experimental setup involves deploying attacker and target nodes on two public servers and we disabled IP-based admission limits on the targets. Attack nodes were instantiated to cover slots in the target discovery table and repeatedly sent unsolicited `Ping` messages to maximize occupancy and evict existing entries. Upon receiving a target's `FindNode`, attackers should reply with `Neighbors` containing attacker entries.

Measurement of DB growth. Since the number of benign nodes in the target's DB directly affects the success of DB stuffing, we deployed a Sepolia full node on a public network server and monitored its DB growth from an empty state. As shown in Fig. 2a, the DB grows at an average rate of 200 nodes per month. Based on Ethereum's official statistics on May 5, 2025, the total number of Sepolia nodes is estimated to be around 2,000. Thus, the maximum realistic DB size on Sepolia is expected to reach 2,000 entries.

Feasibility of poisoning. We have two steps. *Initial filling*: we evaluate whether attacker-controlled nodes can be successfully inserted into an initially empty db. *Repeated insertions*: we evaluate the feasibility of sustaining and expanding DB presence across multiple rounds of poisoning, even after benign entries are added.

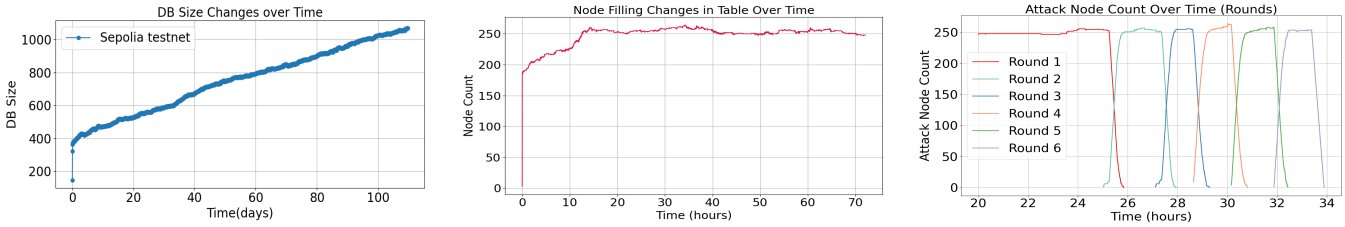
Upon the target's restart, the attacker launched 272 malicious nodes that continuously `Ping` the target to populate its discovery table; we measured the filling rate over time. The first round steadily increased attacker-controlled entries to 95% occupancy within 24 hours (Fig. 2b, Table 1). After terminating that batch, we repeated the procedure with new batches every 2 hours for a total of six rounds. Each new batch was able to refill the table—typically to nearly 100%—within about 1 hour of the previous batch stopping, demonstrating the effectiveness and repeatability of the multi-round filling strategy (Fig. 2c, Table 1).

Table 1: Node Filling Rates for Discovery Table

Round	Experiment times					
	1	**2**	**3**	**4**	**5**	**6**
1st	262/272 (96.32%)	249/272 (91.54%)	255/272 (93.75%)	265/272 (97.43%)	257/272 (94.49%)	260/272 (95.59%)
2nd	256/272 (94.12%)	257/272 (94.49%)	256/272 (94.12%)	264/272 (97.06%)	259/272 (95.22%)	254/272 (93.38%)

Discovery table poisoning. From §2.2, it is known that the division of buckets is based on the number of leading zero bits of the XOR distance of node IDs. The probability P_k that a randomly generated ID falls into a certain bucket k is: $P_k = \frac{1}{2^{16-k}} - \frac{1}{2^{16-(k-1)}} = \frac{1}{2^{17-k}}$.

The probability that the target node selects peers from the last N buckets when establishing outgoing connections is summarized



(a) DB Size Changes over Time **(b) Node Filling Changes over Time** **(c) Attack Node Count over Time**

Figure 2: Evaluations on Sepolia Testnet

in Table 2. Accordingly, we evaluate the discovery table filling rate based on the occupancy ratio of the last five buckets, as these have the highest likelihood of being used for outgoing connections.

We adjust the number of nodes filled in each round. After experimentation, we observe that the seed nodes selected *randomly* from the DB are typically placed in the last 8 buckets of the discovery table. As a result, subsequent rounds of node filling focus only on these 8 buckets, with each round inserting 128 (= 816) nodes.

Ethereum nodes establish outgoing connections by randomly generating target node IDs and selecting the closest matching entries from discovery table. Therefore, increasing the occupancy of higher-indexed buckets (i.e., those farther from local node IDs) directly improves the likelihood that outgoing connections are hijacked by attacker-controlled nodes.

We conduct experiments with two initial DB configurations (257 and 517 benign seed nodes) and observe consistent results, summarized in Table 2. Below, we describe the impact of DB filling attacks using the 257-node case.

Without any DB filling, the last 2 buckets of discovery table remain empty (0% fill rate), and the last 5 buckets are only 22% filled. As DB filling rate increases from 33% to 66%:

- Without altering attacker node behavior, the last two buckets' fill rate increases from 37% to 70%, and the last five buckets from 28% to 44%.
- When attacker node behavior is adjusted (e.g., ID tuning), the fill rate of the last two buckets increases from 36% to 68%, and the last five buckets from 22% to 76%.

These results show that: (i) Standard DB filling effectively improves the occupancy of the last two buckets. (ii) DB filling with attacker behavior adaptation significantly boosts the occupancy of the last five buckets, thereby enhancing the success of outgoing connection hijacking.

4.2 Evaluating Step-② (DNS List)

We evaluate the feasibility and cost of DNS list poisoning attacks. We first locate the identity information of official devp2p nodes to locate the crawler nodes for the DNS list. Then, we estimate the maximum time required to fill the DNS list with nodes under our control, providing a worst-case evaluation of the attack cost.

Locate the identity information of devp2p nodes. Following §3.3, we ran five Ethereum mainnet nodes on a public server and analyzed logs. From these logs we identified the devp2p crawler IP as *34.243.109.22*. This crawler uses a randomized key pair and UDP port at each startup, as confirmed by repeated observations.

Estimation of DNS list filling time. Considering the devp2p tool management in §2.4, to maximize visibility and scoring, our objective is to ensure that attacker-controlled nodes are consistently online and discoverable, receiving the maximum 3-point score during every crawler session. This allows us to estimate the minimum time required to fill the DNS list under ideal conditions.

To uphold ethical standards, we did not perform filling attacks on the official devp2p crawler.Instead we built a minimal controlled testbed on two public servers: one runs the target node (no IP restrictions, generates a random node ID and UDP port at each startup, and uses the latest DNS summary as crawled-node input), and the other hosts five attacker nodes. The attackers continuously `Ping` the target and, upon `FindNode` requests, reply with `Neighbors` messages listing all five attacker entries to maximize exposure. The experiment was repeated for 10 trials.

In 5 out of 10 experiments, at least some attacker nodes successfully entered the target's discovery table. In one experiment, all five attacker nodes reached the maximum score (3 points), demonstrating the feasibility of DNS list poisoning under controlled conditions.

We analyze the score distribution of nodes in the Sepolia testnet DNS list as of July 17, with scores ranging from 275 to 2,688. Assuming each crawl adds only one point to benign nodes and that an attacker can earn 3 points per crawl (a net gain of 2 points), the average daily score advantage is 5 (= $(30/10 - 1) * 5/2$) points. Based on this, we estimate the time required to replace 0%, 25%, 50%, 75%, and 100% of DNS list entries, as shown in Table 3.

4.3 Evaluating Step-③ (Available Slots)

We analyzed the number and distribution of available incoming connection slots on both Ethereum mainnet and the Sepolia testnet, and launched a large-scale slots occupation attack targeting all reachable nodes on Sepolia. We probed more than 2,000 public Ethereum nodes and discovered that over 80% limit available incoming slots to 10 or fewer, supporting the feasibility of large-scale slot occupation. (details in Appendix E).

On July 17, 2025, we screened a total of 1,268 nodes on Sepolia testnet from the official DNS list, measured whether these nodes had available slots. We detected 321 nodes with available slots. We started 200 Sepolia testnet nodes on a public network server and configured all 1,268 nodes as static nodes.

We tried to occupy the incoming available connection slots of nodes in the whole network in real world. After two hours, about

Table 2: Poisoning Discovery Table by DB Pre-filling (*init benign seeds*: number of benign seed nodes initially in the target's DB. *pre-restart DB*: avg. DB fill and seed occupation (%) over 10 runs before restart. *post-restart fill*: avg. fill rate of the last 2, 5, 8, and 17 buckets after restart (10 runs). *outbound selection rate*: prob. that an outbound connection chooses a peer from those buckets. *behavior changed?*: whether attacker behavior was altered. *with/without DB poisoning*: whether step-① was applied.)

init benign seed			DB fill	seed (%)		last-2		last-5		last-8		last-17	
with DB poisoning	257	pre-restart DB	33%	38%	post-restart fill	36%	37%	47%	28%	60%	44%	75%	62%
			50%	58%		53%	55%	60%	34%	69%	47%	81%	65%
			60%	68%		64%	65%	77%	40%	84%	52%	90%	67%
			66%	73%		68%	70%	76%	44%	84%	57%	92%	74%
	517		33%	42%		40%	42%	43%	34%	55%	48%	67%	64%
			50%	56%		58%	60%	65%	43%	74%	51%	79%	64%
			60%	64%		68%	70%	71%	49%	88%	56%	80%	67%
			66%	72%		76%	71%	83%	51%	88%	56%	89%	67%
without	-		0%	0%		0%	0%	22%	22%	41%	41%	57%	57%
		behavior changed?				Yes	No	Yes	No	Yes	No	Yes	No
		outbound selection rate				75%		96.9%		99.6%		100%	

90% of the nodes with available slots were occupied, leaving 34 nodes still with available slots. This means that an available slots occupation attack is feasible on Sepolia testnet.

4.4 Evaluating Step-④ (Outgoing Connections)

We evaluate the effectiveness of outgoing connection hijacking under varying DNS list and DB filling conditions. We conduct experiments in two conditions: before and after network-wide available slots occupation, to assess the impact on hijacking success.

- *DNS list configuration.* Since DNS list filling takes time in practice, we simulate an n% poisoning of Sepolia's 250-node DNS list by replacing the lowest-scored $250 \times n$% entries with attacker-controlled nodes.
- *DB filling configuration.* The target node is initialized with 250 seed nodes, and we perform two rounds of DB filling to achieve a 50% DB poisoning rate.

Before available slots occupation. We conduct 20 experiments on the Sepolia testnet under DNS list poisoning rates of 0%, 25%, 50%, 75%, and 100%, with the DB filling rate fixed at 50%. Table 4 presents the results. As expected, the success rate of outgoing connection hijacking increases as the DNS list poisoning rate rises.

We also analyzed the source of benign connections in the 56 experiments where the target node's incoming connections were not fully occupied. We found that, when the DB filling rate is 50%, 13 of these experiments (23%) had benign connections originating from the discovery table, while 51 experiments (91%) had them from the DNS list.

Table 3: Estimated Time to Reach DNS List Completion under Different Filling Rates

Filling rate	0%	25%	50%	75%	100%
Score	275	325	506	831	2688
Time (days)	54	64	100	166	538

These results indicate that a 50% DB filling rate can effectively limit the target node's ability to establish benign connections via the discovery table.

After available slots occupation. We evaluated outgoing connections hijacking based on the available slots occupation in §4.3. Experimental results are shown in Table 4.

In the case where the DNS list filling rate is 25% and the DB filling rate is 50%, the success rate of outgoing connections after occupying available slots increased by 10%; in comparison, when the DNS list filling rate is 50% and the DB filling rate is 50%, the success rate of outgoing connections after occupying available slots has increased by 50%. The improvement effect is quite significant.

Table 4: Success Rates of Outgoing Connection Hijacking *before* and *after* Available Slots Occupation (FR: filling rate)

DNS list FR	DB FR	*beofore*	*after*
0%	50%	0/20 (0%)	/
25%	50%	4/20 (20%)	6/20 (30%)
50%	50%	8/20 (40%)	19/20 (95%)
75%	50%	13/20 (65%)	/
100%	50%	19/20 (95%)	/

4.5 Evaluating Step-⑤ (Incoming Connections)

We then evaluated attacks on both Sepolia testnet and Ethereum mainnet by monitoring if an attacker could fully occupy incoming connection slots after target nodes restarting.

We first start the attack nodes and configure the target nodes as static nodes, and then start the target nodes. The experiment results are showed in Table 5.

- On the Sepolia testnet, we conducted 30 trials. In all cases (100%), attacker nodes successfully hijacked all incoming connections of the target nodes within 30 seconds of restart.
- On mainnet, we conducted 40 trials. In 24 of them (60%), attackers fully occupied all incoming connection slots within two days of the target's restart. Among these 24 successful trials, 18 (45% of the total) achieved full occupation within 30 seconds of restart.

Table 5: Incoming Connection Hijacking

Network	Success rates		Restart interval of target nodes
Sepolia	30/30 (100%)	30/30 (100%)	within 30s
mainnet	24/40 (60%)	18/40 (45%)	within 30s
		23/40 (57%)	within 1 day
		24/40 (60%)	within 2 days

5 Further Discussion

5.1 Implications of Version Update Frequency

The frequency of Ethereum client updates, particularly for Geth[2], directly impacts how often nodes are restarted, which is an event that can facilitate eclipse attacks due to temporary loss of benign connections. To assess this, we analyzed the release history of Geth from Ethereum's Merge (Sep. 15, 2022) to date (May 5, 2025), spanning over 32 months. Within this period, 56 versions were released, averaging approximately 1.75 per month. This relatively high release cadence implies that mainnet nodes undergo frequent restarts, as each update generally necessitates one, temporarily severing all active peer connections.

This presents a recurring window of opportunity for attackers to exploit, as the node becomes momentarily vulnerable to peer table manipulation and connection hijacking, thereby increasing the feasibility of our proposed eclipse attack.

5.2 IP Resources Required for Our Attack

We estimate the public IP resources needed to perform each step of the proposed eclipse attack.

① *DB pre-filling.* Sepolia testnet contains about 2,000 active nodes (§4.1), The Sepolia testnet has a node size of approximately 2000 nodes, and attackers require 208 public IP addresses with a DB fill rate of 50%. If the attacker wants to achieve a 50% DB fill rate, they need to fill about 2000 nodes in 16 rounds, each round containing 128 nodes (the discovery table allows up to 10 IP addresses from the same/24 subnet, so 208 (= $13 * 16$) public IP addresses are required, each IP running 10 nodes with different ports enabled for each node). For the mainnet, which includes roughly 6,000 nodes (§E), the attacker would need 624 public IP addresses (i.e., 208×3) under the same 50% filling rate assumption.

② *DNS list poisoning.* Due to the constraint that no more than 10 IPs from the same /24 subnet can appear in the discovery table, at least $272/10 = 28$ distinct public IP addresses are required.

③ *Available slots occupation.* To fill the available incoming slots across the network, the attacker needs to control 200 active nodes. A node rejects repeated incoming connections from the same IP within 30 seconds (§2.3). Due to limited IP resources (§4.3), all attack nodes were deployed using a single public IP. However, assuming no IP constraints, 200 unique public IP addresses would allow for rapid occupation of all available slots.

④ *Outgoing connections hijacking.* Similar to the DNS filling step, this step also requires 28 public IP addresses due to the same /24 subnet restriction (272/10).

⑤ *Incoming connections hijacking.* Each Ethereum node allows up to 34 incoming connections by default. In our experiment, we use 40 attack nodes to ensure full occupation. To prevent 30-second rejection due to repeated IPs, each node requires a unique public IP address—thus, 40 public IPs are needed for this step.

Among the five steps, Step-① and Step-③ can reuse the same set of public IP addresses. We estimate the total number of distinct public IPs required for the eclipse attack as follows :

- **Sepolia testnet**: 208 + 28 + 28 + 40 = **304** public IPs;
- **mainnet**: 624 + 28 + 28 + 40 = **720** public IPs.

[2]See: https://github.com/ethereum/go-ethereum/releases/

6 Countermeasures

6.1 Against the Adversarial DB Filling

Establish a blacklist mechanism for high-frequency `pings`. A node records the timestamps of `ping` messages received from each peer. If a peer sends more than 5 `pings` within 1 minute, it is blacklisted, removed from the discovery table, and its UDP messages are dropped. The initial blacklist duration is 1 minute; repeated violations double the duration (2, 4, 8 minutes, etc.) up to a maximum cap (e.g., 24 hours).

6.2 Against the DNS List Poisoning

At present, there are no restrictions on IP resources in the DNS list, and we believe that this type of attack can be mitigated by limiting the attacker's resources.

- **Limit the number of nodes with the same IP address in DNS list.** This requires a large amount of IP resources for attackers to effectively carry out DNS list stuffing attacks.
- **Establish a DNS node blacklist mechanism.** Ethereum can introduce a community-driven reporting system where users submit suspicious DNS nodes along with log evidence. Once the same node receives at least N independent reports, it is added to a DNS blacklist. This threshold mechanism reduces false positives, while ensuring that malicious nodes are gradually excluded from the DNS list.

7 Conclusion

We presented the first practical eclipse attack on Ethereum's peer-to-peer network after the Merge. Our attack enables full control over targeted execution-layer nodes' incoming and outgoing connections. We evaluated on both testnet (Sepolia) and mainnet. We show that an attacker with 304 public IP addresses can achieve a 95% success rate in isolating a target node on Sepolia testnet shortly after its restart. We provided actionable countermeasures.

We also made responsible disclosures to Ethereum. They acknowledged our reported problem and are committed to improving the P2P layer, though no final fix has been released yet.

Acknowledgments

This work was supported by National Natural Science Foundation of China under Grant No.62572070, Intelligent Policing Key Laboratory of Sichuan Province (Grant No.ZNJW2025KFMS003), BUPT innovation and entrepreneurship support program (No.2025-YC-T021/T020), the Open Research Fund of The State Key Laboratory of Blockchain and Data Security, Zhejiang University, the Beijing Natural Science Foundation under Grant No.QY25334, the 111 Project (Grant No.B21049). Dr. Qin Wang was not financially supported.

References

[1] pettinarip, wackerow, bskrksyp9, et al. The merge. *Retrieved by July 2025. https://ethereum.org/en/roadmap/merge/*, 2025.

[2] wackerow, yiannisbot, corwintines, et al. Nodes and clients. *Retrieved by July 2025. https://ethereum.org/en/developers/docs/nodes-and-clients/*, 2025.

[3] Apostolaki M, Zohar A, and Vanbever L. Hijacking bitcoin: Routing attacks on cryptocurrencies. *IEEE Symposium on Security and Privacy (SP)*, pages 375–392, 2017.

[4] Minh Tran, Inho Choi, Gue Joon Moon, and Min Suk Kang. A stealthier partitioning attack against bitcoin peer-to-peer network. *IEEE Symposium on Security and Privacy (SP)*, pages 894–909, 2020.

[5] Baek S, Tran M, and Kang M S. On the sustainability of bitcoin partitioning attacks. *Financial Cryptography and Data Security (FC)*), 2023.

[6] Saad M and Mohaisen D. Three birds with one stone: Efficient partitioning attacks on interdependent cryptocurrency networks. *IEEE Symposium on Security and Privacy (SP)*, pages 111–125, 2023.

[7] Alex Biryukov, Dmitry Khovratovich, and Ivan Pustogarov. Deanonymisation of clients in Bitcoin P2P network. *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 15–29, 2014.

[8] Giulia Fanti and Pramod Viswanath. Deanonymization in the bitcoin p2p network. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.

[9] Alex Biryukov and Sergey Tikhomirov. Deanonymization and linkability of cryptocurrency transactions based on network analysis. *IEEE European Symposium on Security and Privacy (EuroSP)*, pages 172–184, 2019.

[10] Chuyuan Feng and Ju Niu. Selfish mining in Ethereum. *IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 1306–1316, 2019.

[11] Fabian Ritz and Alf Zugenmaier. The impact of uncle rewards on selfish mining in ethereum. In *2018 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 50–57, 2018.

[12] Hongda Kang, Xiang Chang, Ren Yang, Kun Liang, Wei Yu, and Xiapu Luo. Understanding selfish mining in imperfect bitcoin and ethereum networks with extended forks. *IEEE Transactions on Network and Service Management*, 18(3):3079–3091, 2021.

[13] Tanya Tiwari, David Starobinski, and Ari Trachtenberg. Distributed web mining of ethereum. In *International Symposium on Cyber Security Cryptography and Machine Learning (CSCML)*, pages 38–54, 2018.

[14] T. Chen, Z. Li, Y. Zhu, et al. Understanding ethereum via graph analysis. *ACM Transactions on Internet Technology (TOIT)*, 20(2):1–32, 2020.

[15] Huashan Chen, Marcus Pendleton, Laurent Njilla, and Shouhuai Xu. A survey on Ethereum systems security: Vulnerabilities, attacks, and defenses. *ACM Computing Surveys (CSUR)*, 53(3):1–43, 2020.

[16] Chenyu Li, Ren Zhang, and Xiaorui Gong. A place for everyone vs everyone in its place: Measuring and attacking the ethereum global network. *Cryptology ePrint Archive*, 2025.

[17] Kim S K, Ma Z, Murali S, et al. Measuring ethereum network peers. *Proceedings of the Internet Measurement Conference (IMC)*, pages 91–104, 2018.

[18] Lioba Heimbach, Yann Vonlanthen, Juan Villacis, Lucianna Kiffer, and Roger Wattenhofer. Deanonymizing Ethereum validators: The p2p network has a privacy issue. *USENIX Security Symposium (USENIX Sec)*, 2025.

[19] Marcus Y, Heilman E, and Goldberg S. Low-resource eclipse attacks on Ethereum's peer-to-peer network. *Cryptology ePrint Archive*, 2018.

[20] Dahlke G and Eisenbarth T. Low-resource eclipse attacks on alternative ethereum clients. *IEEE European Symposium on Security and Privacy Workshops (EuroSPW)*, pages 45–52, 2018.

[21] Henningsen S, Teunis D, Florian M, et al. Eclipsing Ethereum peers with false friends. *IEEE European Symposium on Security and Privacy Workshops (EuroSPW)*, 2019.

[22] Wüst K and Gervais A. Ethereum eclipse attacks. Technical report, ETH Zurich, 2016.

[23] Heo H, Woo S, Yoon T, et al. Partitioning ethereum without eclipsing it. In *Network and Distributed System Security (NDSS) Symposium*, 2023.

[24] Gencer A E, Basu S, Eyal I, et al. Decentralization in bitcoin and ethereum networks. *International Conference on Financial Cryptography and Data Security (FC)*, pages 439–457, 2018.

[25] Maeng S H, Essaid M, and Ju H T. Analysis of Ethereum network properties and behavior of influential nodes. *Asia-Pacific Network Operations and Management Symposium (APNOMS)*, pages 203–207, 2020.

[26] Masoud M Z, Jaradat Y, Manasrah A, et al. A measurement study of the ethereum underlying P2P network. *Computers, Materials & Continua*, 78(1), 2024.

[27] Lee X T, Khan A, Sen Gupta S, et al. Measurements, analyses, and insights on the entire Ethereum blockchain network, 2020.

[28] Kiffer L, Salman A, Levin D, et al. Under the hood of the Ethereum gossip protocol, 2021.

[29] Eisenbarth J P, Cholez T, and Perrin O. Ethereum's peer-to-peer network monitoring and Sybil attack prevention. *Journal of Network and Systems Management*, 30(4):65, 2022.

[30] Gao Y, Shi J, Wang X, et al. Topology measurement and analysis on Ethereum P2P network, 2019.

[31] Li K, Tang Y, Chen J, et al. Toposhot: Uncovering Ethereum's network topology leveraging replacement transactions, 2021.

[32] Zhao C, Zhou Y, Zhang S, et al. Dethna: Accurate ethereum network topology discovery with marked transactions, 2024.

[33] Pedro Silva, David Vavricka, João Barreto, and Luís Rodrigues. Impact of geo-distribution and mining pools on blockchains: A study of ethereum. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 245–252, 2020.

[34] Gao Y, Shi J, Wang X, et al. Practical deanonymization attack in ethereum based on p2p network analysis, 2021.

[35] Klusman R and Dijkhuizen T. Deanonymisation in Ethereum using existing methods for Bitcoin. *Retrieved October*, 5:2017–2018, 2023.

[36] Béres F, Seres I A, Benczúr A A, et al. Blockchain is watching you: Profiling and deanonymizing Ethereum users, 2021.

[37] Dan Lin, Jiajing Wu, Yunmei Yu, Qishuang Fu, Zibin Zheng, and Changlin Yang. Denseflow: Spotting cryptocurrency money laundering in ethereum transaction graphs. *Proceedings of the ACM Web Conference (WWW)*, pages 4429–4438, 2024.

[38] Wang S, Yang M, Dai W, et al. Deanonymizing ethereum users behind third-party rpc services. *IEEE Conference on Computer Communications (INFOCOM)*, pages 1701–1710, 2024.

[39] Jiaming Zhou, Chao Hu, Jing Chi, et al. Behavior-aware account de-anonymization on ethereum interaction graph. *IEEE Transactions on Information Forensics and Security*, 17:3433–3448, 2022.

[40] W. Chan and A. Olmsted. Ethereum transaction graph analysis. In *2017 12th International Conference for Internet Technology and Secured Transactions (ICITST)*, pages 498–500, 2017.

[41] Heilman E, Kendler A, Zohar A, and Goldberg S. Eclipse attacks on bitcoin's peer-to-peer network. In *USENIX Security Symposium (USENIX Sec)*, pages 129–144, 2015.

[42] Tran M, Shenoi A, and Kang M S. On the routing-aware peering against network-eclipse attacks in bitcoin. *USENIX Security Symposium (USENIX Sec)*, pages 1253–1270, 2021.

[43] Yves-Christian A. E., Bassem Hammi, Ahmed Serhrouchni, et al. Total eclipse: How to completely isolate a bitcoin peer. *Proceedings of the 2018 Third International Conference on Security of Smart Cities, Industrial Control System and Communications (SSIC)*, pages 1–7, 2018.

[44] Shi R, Peng Z, Lan L, et al. Eclipse attacks on monero's peer-to-peer network. *Network and Distributed System Security (NDSS) Symposium*, 2025.

[45] Go Ethereum Team. Fuel depot (v1.14.3). *Retrieved by July 2025. https://github.com/ethereum/go-ethereum/releases/tag/v1.14.3*, 2024.

[46] wackerow, chainide-agent-bob, m4sterbunny, et al. Node architecture. *Retrieved by July 2025. https://ethereum.org/en/developers/docs/nodes-and-clients/node-architecture/*, 2025.

[47] fjl, subtly, FrankSzendzielarz, et al. Node discovery protocol. *Retrieved by July 2025. https://github.com/ethereum/devp2p/blob/master/discv4.md*, 2023.

[48] Saad M, Chen S, and Mohaisen D. Syncattack: Double-spending in Bitcoin without mining power. *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1668–1685, 2021.

Ethics Considerations

All the eclipse attack experiments involved in this study were conducted in a strictly controlled environment and did not have any impact on mainnet or any actual operating systems. This includes technical means such as DNS list poisoning, connection hijacking, and DB pre-filling (our attacking steps ①-⑤), which are only applied to the target nodes we independently built. We have not conducted any unauthorized access, denial of service attacks, or caused interference to third-party nodes. Our experiments is to evaluate the feasibility and potential risks of eclipse attacks in a controlled environment. All data collected during the research process, such as node identity, connectivity behavior, and rating changes, are sourced from publicly observable information or controlled interactions and do not involve the collection of any sensitive information.

In accordance with responsible disclosure practices, we have submitted a full vulnerability report to the Ethereum official team. The report includes detailed descriptions of the attack scenarios, reproduction steps, and our proposed fixes.

We believe that responsible research on security threats is crucial for enhancing the robustness of decentralized systems.

A Related Work

Eclipse attacks against Ethereum. Marcus*et al.* [19] demonstrated a low-resource eclipse on Geth v1.6.6: with only two attacker hosts they could monopolize a target's incoming and outgoing connections by exploiting (1) the lack of a post-restart incoming-connection cap and (2) the fact that Geth's UDP listener becomes active before bootstrapping, allowing unsolicited Ping-based connections. Dahlke *et al.* [20] showed the method also applies to Parity. Henningsen *et al.* [21] proposed the "False Friends" attack (Geth v1.8.0), using two IPs from different /24 subnets to occupy incoming slots without requiring a restart, and noting that Geth's outgoing selection (one head per 17 buckets) can be abused by unsolicited Pings to seize outgoing slots.

Partition attacks against Ethereum. Wüst *et al.* [22] proposed a partitioning attack on Ethereum in 2016, exploiting weaknesses in its proof-of-work mechanism. The attacker creates a fork with more blocks but lower total difficulty and manipulates difficulty reports to mislead the victim into syncing with the malicious chain. By injecting a forged block with minimal difficulty, the attacker raises the victim's difficulty threshold, causing it to reject valid blocks from the honest network and remain isolated. Heo *et al.* [23] introduced the *Gethlighting* attack, a new partitioning technique that significantly lowers the resource requirements. Unlike prior attacks that require full control over peer connections, Gethlighting leverages a low-rate denial-of-service (DoS) strategy. By intermittently disrupting the target node's operation, the attacker is able to delay blockchain synchronization for several hours, effectively partitioning the node from the Ethereum network.

Eclipse attacks and more. Heilman *et al.* [41] were the first to introduce the concept of the eclipse attack in the context of Bitcoin. Their method involves populating the target node's peer table to monopolize its outgoing connections. Tran *et al.* [42] proposed a more stealthy variant, termed *Erebus*, which leverages the capabilities of autonomous system (AS)-level adversaries. By populating the target's peerlist with IP addresses owned by a malicious AS and intercepting traffic at the network layer, the attacker gradually positions itself as a persistent intermediary, ultimately gaining control over all of the target's connections. Saad *et al.* [48] first introduced the idea of occupying idle connection slots across the network. However, they did not provide experimental validation on the Bitcoin network. Shi *et al.* [44] extended eclipse attack to Monero's network. They proposed a connection reset attack, where the attacker actively forces the target node to drop existing benign connections without requiring a restart. This enables malicious peers to rapidly seize those freed slots and establish control over the target's peer set.

B The Lookup Recursive Search Process of Node Filling

The node lookup of the discovery table is an iterative process. The recursive search algorithm process is as follows:

i) Randomly generate a node ID;
ii) Find the 16 nodes closest to the ID from the node list, with a set of N_0 nodes. Sort them in order of their distance from the node ID in i), from near to far;
iii) In N_0, in the order of ii), select a node that has not been queried (and has not sent a `FindNode` message to that node), send a `FindNode` message, label the node as queried, and merge the 16 nodes in the received `Neighbors` messages with N_0. Find the 16 nodes closest to the ID from the merge and replace the nodes in N_0. The nodes in the received neighbor messages are stored in the discovery table of the local node;
v) Repeat the iteration of iii) until all nodes in N_0 have been queried, and the iteration ends.

C The Reasons why Incoming Connection Attempts fail

First, if the node has already reached its incoming connection limit, new requests are automatically denied. Second, the node enforces a rate-limiting policy that rejects all incoming connections from the same IP address within a 30-second interval to prevent repeated rapid attempts. Finally, connections may be rejected due to protocol-level issues, such as mismatched network IDs or fork IDs, receipt of malformed or excessively long handshake messages (e.g., during the ETH or Snap protocol negotiation), duplicate connection attempts, or connections from the same peer ID.

Further details on these rejection conditions can be found in the handshake failure logic of the RLPX[3], ETH[4], and Snap[5].

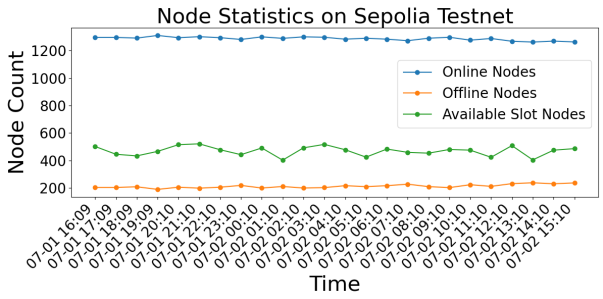
D Selecting Outgoing Connections

There are two outgoing connection selection resources for an Ethereum node, namely, lookup discovery and DNS discovery. We will introduce the two node-discovery methods below.

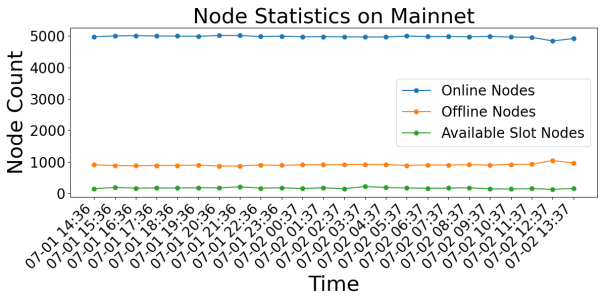
Node source based on lookup discovery. The node source based on lookup discovery can cache up to 16 nodes. When the node source is empty, the local node will enable lookup discovery. Each lookup discovery targets the same target ID and requires multiple iterations. Below is an introduction to the iterative process of each lookup. At the beginning of each lookup, the first step is to determine whether the local list has been searched:

- *No*, then this iteration is the initial iteration round for this lookup discovery. A node ID is randomly generated as the target ID for this lookup discovery, and the 16 nodes closest to the target ID are selected from the local node list. They are sorted from small to large according to their distance from the target ID and added to the cache list result_entries for this lookup discovery. Meanwhile, these 16 nodes are added to the `lookup_buffer` and marked as seen. This round of iteration ends.
- *Yes*, select a node from the cache list that has not been queried in order, send it a `FindNode` message, and receive `Neighbors` messages. Combine the 16 nodes in the `Neighbors` messages with the nodes in the cache list, retain the 16 nodes closest to the target ID, and sort them from small to large according to their

[3]RLPX protocol handshake failure: https://github.com/ethereum/go-ethereum/blob/v1.14.3/p2p/peer_error.go#L75
[4]ETH protocol handshake failure: https://github.com/ethereum/go-ethereum/blob/v1.14.3/eth/protocols/eth/protocol.go#L67
[5]Snap protocol handshake failure section, refer to https://github.com/ethereum/go-ethereum/blob/v1.14.3/eth/protocols/snap/protocol.go#L59.

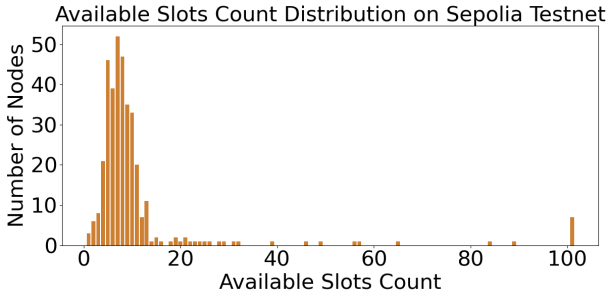


(a) Node Statistics on Sepolia Testnet

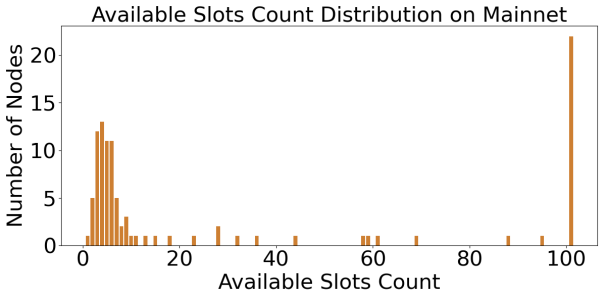


(b) Node Statistics on Mainnet

Figure 3: Node Statistics in Ethereum P2P Network



(a) Available Slots Count Distribution on Sepolia Testnet



(b) Available Slots Count Distribution on Mainnet

Figure 4: Available Slots Count in Ethereum P2P Network

distance from the target ID. Meanwhile, these 16 nodes are taken from nodes that are not in the list, added to the `lookup_buffer`, and marked as seen. This round of iteration ends.

The end condition for each lookup is that all nodes in the cache list searched in this lookup have been queried.

Node source based on DNS discovery. After the local node is started and initialized in the node list, it will retrieve nodes from the URL of the candidate node list discovered based on DNS and store them all in the `dns_buffer` buffer.

The DNS discovery process is as follows:

- *Obtaining DNS seeds.* In the source code of the Ethereum client, a specified DNS domain name is hard coded and used as a DNS seed to initiate the node discovery process;
- *Query DNS records and construct a node tree.* Local nodes recursively query subdomains based on branch information provided in the DNS domain name until all leaf node ENR records are obtained, and construct a DNS node tree.
- *Select a node to establish a TCP connection.* The local node extracts the node enr record from the node tree and extracts information such as node ID, IP, TCP port, etc. from the enr record. The local node attempts to establish a TCP connection with the node based on this information.

E Available Slots Measurement in Ethereum P2P Network

Measurement of nodes with available slots. To measure the availability of inbound connection slots, we extracted node sets for Ethereum mainnet and the Sepolia testnet from DNS lists publicly maintained by Ethereum (snapshot taken at 09:50 UTC on July 1st, 2025). Our probing program, deployed on a cloud server in the United States, monitored these nodes continuously over a 24-hour period. Each node was scanned hourly to determine its online status and the number of available incoming connection slots.

We report the average results across 24 hourly scans in Table 6. Each scan cycle took approximately 11 minutes for the mainnet node set and 6 minutes for Sepolia. The temporal variation in the number of nodes with available slots is in Fig. 3.

We observe that the online rate of nodes on both Sepolia testnet and mainnet is consistently high, at approximately 85%. However, there exists a stark contrast in the proportion of nodes with available inbound connection slots: around 30% of Sepolia nodes have available slots, compared to only about 3% on mainnet. This indicates that while overall node availability is similar, Sepolia exhibits significantly greater slot openness.

Distribution of available slots quantity. We launched 100 attack nodes on a public network server to actively occupy the idle connection slots of Ethereum nodes with available slots. Additionally, a dedicated detection node was deployed to verify whether all available slots had been successfully filled.

Table 6: Measurement of Nodes with Available Incoming Connection Slots in the Entire Network

No.	Network	Average number of nodes		
		Online	Offline	with available slots
1500	Sepolia	1288 (85.9%)	212 (14.1%)	469 (31.3%)
5887	mainnet	4979 (84.6%)	908 (15.4%)	168 (2.8%)

No.: number of detected nodes
available slots: the available incoming connection slots

The measurement results are summarized in Table 7 and Fig. 4. It can be seen from the table that the number of nodes with less than 10 available slots on Sepolia testnet and mainnet accounts for the majority of the number of available slot nodes, which are 80.1% and 62.7% respectively.

Table 7: Available Slots Distribution in Ethereum Nodes

No.	Network	Distribution of available slots			
		1-10	11-50	51-99	100+
362	Sepolia	290 (80.1%)	60 (16.6%)	5 (1.4%)	7 (1.9%)
102	mainnet	64 (62.7%)	10 (9.8%)	5 (4.9%)	23 (22.5%)

No.: number of nodes with available slots
Note: The detection duration for both networks was 7 hours.

F Pseudocode for Our Eclipse Attack

We provide implementation details of our attack (cf. Algorithm 1), covering the main logic and several key functions.

In Algorithm 1, we give the detailed implementation process of the eclipse attack against Ethereum, including the main control logic and several key subprograms.

In the attack initialization phase, two sub threads are started at the same time: one is to execute the `db_pre_filling_attack` on the target node, polluting its local DB; the other is to execute the `dns_list_poisoning_attack` on the official devp2p network crawler node of Ethereum. Next, the attacker continuously monitors whether the target node is online until it detects its restart.

After the target node restarts, the attacker immediately starts three operations: 1) Execute `occupy_available_slots_attack` on all nodes in the network, and try to preempt the incoming connection slots of all active nodes, so that the target node cannot re-establish the connection with the benign nodes when it restarts; 2) Perform the `discovery_table_poisoning_attack` on the target node to ensure that the attacking nodes are dominant in its restarted discovery table; 3) Start to `occupy_in_connections` of the target node and make it connected by attacking nodes.

Upon the above operations are executed, the attack main loop starts to run: continuously obtain the current number of incoming connections and outgoing connections (`get_occupied_incon` and `get_occupied_outcon`) of the target node, and compare them with the maximum number of allowed connections (34 and 16 respectively). As long as the connection slots are still unoccupied, the attacker will continue to launch connection attempts.

When the incoming and outgoing connections of the target node are all controlled by the attacking nodes, it means that the node is completely isolated from the honest network, and the eclipse attack is successfully completed.

Algorithm 1: Our **Eclipse Attack** against Ethereum

```
Input: TargetNode = (target_ip, target_port)
Data: OccupiedInConnections = 0,
OccupiedOutConnections = 0,
AllActiveNodes = nodes in Ethereum Sepolia testnet,
OfficialDevp2pNode = the Ethereum official devp2p crawl node,
N = the rounds of db filling attack,
AttackNodes = our controlled nodes,
MaxInConnectionsOfTargetNode = 34,
MaxOutConnectionsOfTargetNode = 16
StartThread(db_pre_filling_attack(AttackNodes, TargetNode, N))
StartThread(dns_list_poisoning_attack(AttackNodes, OfficialDevp2pNode))
while is_target_online(TargetNode) do
    wait()
StartThread(occupy_available_slots(AttackNodes, AllActiveNodes))
wait_for(TargetNode_reboot())
StartThread(discovery_table_poisoning_attack(AttackNodes,
  TargetNode))
StartThread(occupy_in_connections(AttackNodes, TargetNode))
while OccupiedOutConnections < MaxOutConnectionsOfTargetNode and
  OccupiedInConnections < MaxInConnectionsOfTargetNode do
    OccupiedOutConnections = get_occupied_outcon(TargetNode)
    OccupiedInConnections = get_occupied_incon(TargetNode)
return ok

Subroutines
    discovery_table_poisoning_attack(attack_node_set, target_node)
    Data: param1: attack_node_set, param2: target_node;
    foreach attack_node in attack_node_set do
        attack_node.send_ping(target_node)
        attack_node.wait_pong(target_node)
        attack_node.send_pong(target_node)
        attack_node.wait_findnode(target_node)
        attack_node.send_neighbors(AttackNodes)
        attack_node.wait_enrrequest(target_node)
        attack_node.send_enrresponse(target_node)
    db_pre_filling_attack(attack_node_set, target_node, N)
    Data: param1: attack_node_set, param2: target_node, param3: N;
    for i = 0 in N do
        discovery_table_poisoning_attack(attack_node_set, target_node)
    dns_list_poisoning_attack(attack_node_set, official_node)
    Data: param1: attack_node_set, param2: official_node;
    while true do
        discovery_table_poisoning_attack(attack_node_set,
          official_node)
    occupy_available_slots(attack_node_set, node_set)
    Data: param1: attack_node_set, param2: node_set;
    foreach attack_node in attack_node_set do
        foreach node in node_set do
            attack_node.connect(node)
    occupy_in_connections(attack_node_set, target_node)
    Data: param1: attack_node_set, param2: target_node;
    foreach attack_node in attack_node_set do
        attack_node.connect(target_node)
    get_occupied_outcon(attack_node_set)
    Data: param1: attack_node_set;
    return count of outbound connections to attack_node_set
    get_occupied_incon(attack_node_set)
    Data: param1: attack_node_set;
    return count of inbound connections from attack_node_set
```